Hamming Codes

A Linear Algebra Tutorial

Abstract

A development of Hamming codes through kernels and bases, followed by a study of error detection, correction, and redundancy using Hamming distance and Hamming balls.

Contents

Why Error Detection and Correction Are Needed **2**

1 Representing Bit Strings and Their Positions **2**

2 Turning Position Labels into Parity Checks **3**

3 What the Checks Reveal About an Error **4**

4 Why Every Single-Bit Error Can Be Located **4**

5 Why the Conclusion Requires the Single-Error Assumption **5**

6 Choosing Which Positions Hold Check Digits **6**

7 Measuring Errors with Hamming Weight and Distance **8**

8 Why Nonzero Codewords Are Exactly the Undetectable Error Patterns **9**

9 Why Minimum Distance Equals Minimum Nonzero Weight **11**

10 Reading Minimum Distance from the Columns of the Check Matrix **12**

11 Why Correcting Up to t Errors Requires Minimum Distance Greater Than 2t **13**

12 Hamming Balls as Sets of Possible Received Words **15**

13 A Three-Bit Example Bringing the Arguments Together **18**

14 Word Length, Redundancy, and Correction Regions **20**

Why Error Detection and Correction Are Needed

Binary data can change during storage, reading, or transmission: a bit intended as 0 may become 1, or vice versa. If every bit string of a given length is a valid representation, a corrupted string still looks valid. To detect errors from the received string alone, we need redundancy: constraints that distinguish valid representations from some corrupted ones. Error detection identifies a violation of these constraints; error correction uses the redundancy to recover the original data within a specified range of errors.

Hamming's work grew out of the reliability problems of early computers. An undetected error could invalidate subsequent computation, while detecting an error could still leave the machine unable to continue until the problem was corrected. His 1950 paper, *Error Detecting and Error Correcting Codes*, presented foundational constructions for detecting and correcting errors in binary data. Hamming 1950

How can a small amount of additional check information reveal not only that a bit has flipped, but also which bit? This tutorial develops Hamming codes from kernels and bases, then uses Hamming distance to explain detection, correction, and redundancy.

The prerequisites are vector spaces, linear maps, kernels, bases, invertibility, and the rank-nullity theorem. All vector and matrix arithmetic below takes place in $\mathbb{F}_2$; these familiar linear algebra principles still apply.

1 Representing Bit Strings and Their Positions

We work over the binary field

$$\mathbb{F}_2 = \{0, 1\}, \tag{1.1}$$

where addition and multiplication are performed modulo 2. In particular,

$$1 + 1 = 0.$$

Adding 1 to a bit therefore flips its value.

Let $m \geq 2$ be the number of binary digits in each position label. It will also be the number of independent parity checks and the length of the syndrome. In the Hamming construction, the total word length is

$$n = 2^m - 1. \tag{1.2}$$

A binary word of length n, including both information digits and check digits, is represented by a column vector

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{F}_2^n. \tag{1.3}$$

Number its positions $1, 2, \ldots, n$. Write each position number j in binary using exactly m digits:

$$j = a_1\, 2^{m-1} + a_2\, 2^{m-2} + \cdots + a_m\, 2^0, \qquad a_1, \ldots, a_m \in \{0, 1\}.$$

Assign position j the vector label

$$h_j := \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{pmatrix} \in \mathbb{F}_2^m. \tag{1.4}$$

For example, when $m = 3$,

Position j	1	2	3	4	5	6	7
Binary representation	001	010	011	100	101	110	111

These labels are exactly the nonzero vectors of $\mathbb{F}_2^m$. The ambient vector space includes the zero vector; the set of position labels excludes it.

Notice that a word x belongs to $\mathbb{F}_2^n$, whereas a position label h_j belongs to $\mathbb{F}_2^m$. They are different mathematical objects.

2 Turning Position Labels into Parity Checks

Arrange the labels as the columns of a matrix:

$$H := \begin{pmatrix} | & | & & | \\ h_1 & h_2 & \cdots & h_n \\ | & | & & | \end{pmatrix} \in \mathbb{F}_2^{m \times n}. \tag{2.1}$$

For $m = 3$, this gives

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

Consequently,

$$Hx = \begin{pmatrix} x_4 + x_5 + x_6 + x_7 \\ x_2 + x_3 + x_6 + x_7 \\ x_1 + x_3 + x_5 + x_7 \end{pmatrix}.$$

Each row selects the positions whose labels have a 1 in the corresponding coordinate. The resulting entry is 0 if those bits contain an even number of ones, and 1 if they contain an odd number.

Thus, H turns the position labels into m parity checks.

We define the set of valid words by

$$C := \left\{ c \in \mathbb{F}_2^n : Hc = 0_{\mathbb{F}_2^m} \right\}. \tag{2.2}$$

Equivalently, the code is the kernel of the linear map $H : \mathbb{F}_2^n \to \mathbb{F}_2^m$:

$$C = \ker(H). \tag{2.3}$$

The columns of H include a basis of $\mathbb{F}_2^m$, so $\operatorname{rank}(H) = m$. By rank-nullity,

$$k := \dim C = n - m, \qquad |C| = 2^k = 2^{n-m}. \tag{2.4}$$

Here k is the number of independent information bits, and $|C|$ denotes the number of codewords. Thus a transmitted word has n bits: k information bits and m redundant check bits. For $m = 3$, these quantities are $n = 7$, $k = 4$, and $|C| = 16$.

The agreement is that the encoder sends only words in C. Every transmitted word must pass all m parity checks.

Because the columns of H are exactly all the nonzero vectors of $\mathbb{F}_2^m$, this construction defines the binary Hamming code of length $2^m - 1$.

3 What the Checks Reveal About an Error

Suppose the transmitted word is $c \in C$, and the received word is

$$y = c + e, \tag{3.1}$$

where

$$e = \begin{pmatrix} e_1 \\ \vdots \\ e_n \end{pmatrix} \in \mathbb{F}_2^n$$

is the error pattern: an entry of 1 means that the corresponding bit has flipped.

Define the **syndrome** of the received word by

$$s := Hy. \tag{3.2}$$

By linearity,

$$s = H(c + e) = Hc + He = He, \tag{3.3}$$

because $Hc = 0$.

This is the purpose of imposing the parity constraints: the syndrome removes the valid transmitted word and retains only the effect of the error.

4 Why Every Single-Bit Error Can Be Located

To avoid confusing an error vector with its entries, write u_j for the standard basis vector of $\mathbb{F}_2^n$ whose only nonzero entry is in position j:

$$u_j := \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix} \quad \text{with the 1 in position } j. \tag{4.1}$$

If only bit j flips, then $e = u_j$, and

$$s = Hu_j = \begin{pmatrix} | & | & & | \\ h_1 & h_2 & \cdots & h_n \\ | & | & & | \end{pmatrix} \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix} = h_j. \tag{4.2}$$

Multiplication by u_j selects the j-th column. The syndrome is therefore exactly the label assigned to the flipped position.

This also explains the two requirements on the labels:

- **No label is zero.** Otherwise, a flip at that position would produce syndrome 0, exactly as no error does.

- **No two labels are equal.** Otherwise, single-bit errors at those two positions would produce the same syndrome.

Under the assumption of at most one error, the allowed error patterns are

$$0, u_1, \ldots, u_n.$$

These $n+1$ patterns must have different syndromes. Since $\mathbb{F}_2^m$ contains exactly 2^m vectors, we must have

$$n+1 \leq 2^m, \qquad \text{or equivalently} \qquad n \leq 2^m - 1. \tag{4.3}$$

The Hamming construction reaches equality in (4.3) by using every nonzero vector of $\mathbb{F}_2^m$ exactly once as a column of H. Thus (1.2) gives the largest word length for which m syndrome bits distinguish no error from every single-bit error.

The zero vector is therefore present in the syndrome space but reserved for the no-error case **within the assumption of at most one error**.

Under that assumption, decoding follows the rule

$$\widehat{c} = \begin{cases} y, & s = 0, \\ y + u_j, & s = h_j. \end{cases} \tag{4.4}$$

In the second case,

$$y + u_j = c + u_j + u_j = c.$$

For a concrete example, take

$$c = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}, \qquad Hc = \begin{pmatrix} 0+0+1+1 \\ 1+1+1+1 \\ 0+1+0+1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}. \tag{4.5}$$

If bit 6 flips, then

$$y = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}, \qquad Hy = \begin{pmatrix} 0+0+0+1 \\ 1+1+0+1 \\ 0+1+0+1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = h_6. \tag{4.6}$$

The checks identify position 6, which we flip back.

5 Why the Conclusion Requires the Single-Error Assumption

The general error vector e need not equal one standard basis vector. If two different positions i and j flip, then

$$e = u_i + u_j,$$

and the syndrome becomes

$$s = H(u_i + u_j) = h_i + h_j. \tag{5.1}$$

Since $h_i \neq h_j$, their sum is nonzero. Since H contains every nonzero vector of $\mathbb{F}_2^m$, there is a position ℓ such that

$$h_i + h_j = h_\ell.$$

Thus,

$$H(u_i + u_j) = Hu_\ell. \tag{5.2}$$

A two-bit error and a one-bit error can produce exactly the same syndrome.

For example,

$$h_1 + h_2 = \begin{pmatrix}0\\0\\1\end{pmatrix} + \begin{pmatrix}0\\1\\0\end{pmatrix} = \begin{pmatrix}0\\1\\1\end{pmatrix} = h_3. \tag{5.3}$$

If bits 1 and 2 actually flipped, the single-error rule would incorrectly flip bit 3, producing

$$\widehat{c} = c + u_1 + u_2 + u_3 \neq c.$$

Nevertheless,

$$H\widehat{c} = Hc + h_1 + h_2 + h_3 = 0.$$

The decoder would produce a different valid codeword.

The matrix calculation remains correct. What fails is the inference that the syndrome identifies a unique error once multiple-bit errors are allowed.

We have therefore proved the conditional statement:

> If a valid Hamming codeword experiences at most one bit flip, its syndrome uniquely identifies the error and allows the original word to be recovered.

A nonzero syndrome alone does not establish that only one bit flipped. Likewise, without the error bound, a zero syndrome does not establish that no error occurred.

6 Choosing Which Positions Hold Check Digits

The correction proof used the matrix H and the condition $Hc = 0$. It did not require any particular positions to be designated as check digits.

We can now ask an encoding question: which positions may hold the check digits so that the remaining positions can hold arbitrary information?

Choose m positions as check positions. Let their columns form

$$H_P \in \mathbb{F}_2^{m\times m}.$$

Let the remaining columns form

$$H_D \in \mathbb{F}_2^{m\times(n-m)}.$$

Grouping the coordinates for this calculation, write the corresponding values as

$$p \in \mathbb{F}_2^m, \qquad d \in \mathbb{F}_2^{n-m}.$$

The validity condition becomes

$$\begin{pmatrix} H_P & H_D \end{pmatrix} \begin{pmatrix} p \\ d \end{pmatrix} = 0,$$

or equivalently,

$$H_P p + H_D d = 0. \tag{6.1}$$

Our encoding requirement is:

> For every choice of information digits d, there must be exactly one choice of check digits p that makes the complete word valid.

If the selected labels form a basis of $\mathbb{F}_2^m$, then H_P is invertible. Since subtraction equals addition in $\mathbb{F}_2$,

$$H_P p = H_D d, \qquad p = H_P^{-1} H_D d. \tag{6.2}$$

Every information vector therefore determines exactly one check vector.

Conversely, if the selected labels are linearly dependent, there is a nonzero vector q such that

$$H_P q = 0.$$

For the information vector $d = 0$, both $p = 0$ and $p = q$ satisfy the checks. Uniqueness fails.

Hence:

> A set of m positions can serve as the check-digit positions exactly when their labels form a basis of $\mathbb{F}_2^m$.

For the seven-bit example, positions $1, 2, 4$ have labels

$$\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

They form the standard basis, in reverse order, making them a convenient choice.

We can use this choice to encode an actual message. Keep the transmitted coordinates in their original order, and write

$$d = \begin{pmatrix} c_3 \\ c_5 \\ c_6 \\ c_7 \end{pmatrix}, \qquad p = \begin{pmatrix} c_1 \\ c_2 \\ c_4 \end{pmatrix}. \tag{6.3}$$

The equations $Hc = 0$ determine

$$\begin{aligned} c_1 &= c_3 + c_5 + c_7, \\ c_2 &= c_3 + c_6 + c_7, \\ c_4 &= c_5 + c_6 + c_7. \end{aligned} \tag{6.4}$$

For the message $d = (1, 0, 1, 1)^{\mathsf{T}}$, these equations give

$$p = \begin{pmatrix} 1+0+1 \\ 1+1+1 \\ 0+1+1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}. \tag{6.5}$$

Placing these entries in their designated positions produces

$$c = (0, 1, 1, 0, 0, 1, 1)^{\mathsf{T}}, \tag{6.6}$$

which agrees with (4.5).

This procedure defines a linear encoding map

$$E : \mathbb{F}_2^4 \longrightarrow \mathbb{F}_2^7, \qquad \mathrm{im}(E) = C = \ker(H). \tag{6.7}$$

Every output passes the checks, and every codeword is obtained by encoding its entries in positions $3, 5, 6, 7$. Reading those four positions recovers the message, so E is one-to-one. Thus E gives a linear isomorphism between the message space $\mathbb{F}_2^4$ and the code space C.

For the received word in (4.6), the decoder (4.4) returns (6.6). Reading its information coordinates gives

$$d = (c_3, c_5, c_6, c_7)^{\mathsf{T}} = (1, 0, 1, 1)^{\mathsf{T}}. \tag{6.8}$$

The choice of check positions was convenient rather than necessary. Positions $3, 5, 7$ also work. Their labels satisfy

$$\alpha h_3 + \beta h_5 + \gamma h_7 = \begin{pmatrix} \beta + \gamma \\ \alpha + \gamma \\ \alpha + \beta + \gamma \end{pmatrix}.$$

Setting this equal to zero gives

$$\beta = \gamma, \qquad \alpha = \gamma, \qquad \gamma = 0,$$

and hence $\alpha = \beta = \gamma = 0$. These labels are linearly independent and therefore form another basis.

Changing which of these positions are designated as check digits changes how information is placed into the codeword. With H fixed, it leaves the set of valid codewords and the single-error correction argument unchanged.

7 Measuring Errors with Hamming Weight and Distance

Equation (5.2) shows that different error patterns can have the same syndrome. Hamming weight and distance quantify how many positions are involved.

The first question is simply: how many positions does an error pattern affect?

The **Hamming weight** of a vector $v \in \mathbb{F}_2^n$ is its number of nonzero coordinates:

$$\mathrm{wt}(v) := \#\{j \in \{1, \ldots, n\} : v_j \neq 0\}. \tag{7.1}$$

In examples below, a compact string such as 1000010 denotes a column vector with those entries, read from position 1 on the left to position n on the right. For example,

$$\mathrm{wt}(0000000) = 0, \qquad \mathrm{wt}(1000010) = 2, \qquad \mathrm{wt}(0110011) = 4.$$

Weight is a property of a vector. The valid codeword 0110011 has weight 4 even when transmitted without corruption. Applied to the error vector e, weight counts flipped positions, because $e_j = 1$ exactly when position j has flipped.

We can also count differing positions by comparing two words. For $x, z \in \mathbb{F}_2^n$, define their **Hamming distance** by

$$d_H(x, z) := \#\{j \in \{1, \ldots, n\} : x_j \neq z_j\}. \tag{7.2}$$

At each position, $z_j - x_j$ is nonzero exactly when $x_j \neq z_j$. Therefore,

$$d_H(x, z) = \mathrm{wt}(z - x). \tag{7.3}$$

In particular,

$$\boxed{d_H(c, y) = \mathrm{wt}(y - c) = \mathrm{wt}(e).} \tag{7.4}$$

Distance compares two words; weight counts the nonzero entries of one vector. Taking their difference connects the two.

For the single-bit error in (4.6),

$$c = 0110011, \qquad y = 0110001, \qquad e = y - c = 0000010.$$

The words differ only at position 6, so

$$d_H(c, y) = \mathrm{wt}(e) = 1.$$

More generally, $d_H(x, z)$ is the smallest number of single-bit flips needed to change x into z: every differing position must be flipped at least once, and flipping each such position once achieves the change.

Hamming distance satisfies the usual requirements for a distance. It is nonnegative, equals zero exactly when the two words are identical, and is unchanged when the words are exchanged. It also satisfies the triangle inequality:

$$d_H(x, z) \leq d_H(x, w) + d_H(w, z). \tag{7.5}$$

To see why, consider any position where x and z differ. At that position, w cannot agree with both of them, so that position is counted in at least one of the two distances on the right. Summing over positions proves the inequality.

All of these counts refer to coordinate positions. They do not measure the difference between the integers represented by the bit strings.

8 Why Nonzero Codewords Are Exactly the Undetectable Error Patterns

Recall that the syndrome is

$$s = Hy = He.$$

A nonzero syndrome reveals that the received word violates at least one parity check. A zero syndrome says that the received word passes every check; it does not, by itself, say that the received word is the word that was sent.

In particular, a nonzero error is undetectable by these checks exactly when

$$e \neq 0, \qquad He = 0.$$

Calling a vector a **codeword** says that it satisfies the checks; calling it an **error pattern** describes its role in a transmission. Every $v \in \mathbb{F}_2^n$ can occur as an error pattern: flipping the positions where $v_j = 1$ changes a transmitted word c into $y = c + v$.

Define the nonzero undetectable error patterns by

$$\mathcal{U} := \{e \in \mathbb{F}_2^n : e \neq 0,\ He = 0\}. \tag{8.1}$$

For any $v \in C \setminus \{0\}$, (2.3) gives

$$v \neq 0, \qquad Hv = 0 \quad \Longrightarrow \quad v \in \mathcal{U}.$$

Hence

$$C \setminus \{0\} \subseteq \mathcal{U}. \tag{8.2}$$

For any transmitted $c \in C$, this error produces

$$y = c + v \neq c, \qquad Hy = Hc + Hv = 0.$$

Conversely,

$$\begin{aligned} e \in \mathcal{U} &\Longrightarrow e \neq 0, \quad He = 0 \\ &\Longrightarrow e \in \ker(H) \setminus \{0\} = C \setminus \{0\}. \end{aligned}$$

Therefore,

$$\mathcal{U} \subseteq C \setminus \{0\}. \tag{8.3}$$

The two inclusions give

$$\boxed{\mathcal{U} = C \setminus \{0\}.} \tag{8.4}$$

Equation (5.3) gives

$$h_1 + h_2 + h_3 = 0.$$

Consequently,

$$v = u_1 + u_2 + u_3 = 1110000$$

satisfies $Hv = 0$ and is a nonzero codeword. Take the transmitted word from (4.5):

$$c = 0110011.$$

If v occurs as the error pattern, the received word is

$$y = c + v = 1000011.$$

These words differ at three positions, but

$$Hy = Hc + Hv = 0.$$

Thus v has two legitimate descriptions: it is a valid vector in C, and it is an undetectable error pattern that changes c into another valid word.

We can also describe this situation starting from two codewords. If an undetectable error changes c into a different codeword c', then

$$e = c' - c.$$

Conversely, for any two distinct codewords c, c', the vector $e = c' - c$ is nonzero and satisfies

$$He = Hc' - Hc = 0.$$

The error is invisible because it carries us from one allowed word to another.

We therefore want to know the smallest possible weight of a vector in $\mathcal{U}$. Since $\mathcal{U}$ and $C \setminus \{0\}$ are the same set,

$$\min_{\substack{e \neq 0 \\ He = 0}} \operatorname{wt}(e) = \min_{\substack{v \in C \\ v \neq 0}} \operatorname{wt}(v).$$

The zero vector is excluded because it represents no error; including it would make both minima trivially zero.

This is a minimum weight of errors whose syndrome is zero, not a minimum weight of the syndrome itself. The two weights measure different things. For example,

$$e = u_6 = 0000010, \qquad s = He = h_6 = 110.$$

Here $\mathrm{wt}(e) = 1$ but $\mathrm{wt}(s) = 2$: one flipped bit causes two parity checks to fail.

9 Why Minimum Distance Equals Minimum Nonzero Weight

The **minimum distance** of the code is

$$d_{\min} := \min_{\substack{c,c' \in C \\ c \neq c'}} d_H(c, c'). \tag{9.1}$$

The minimum exists because C is finite and contains at least two codewords.

This definition measures the closest separation of distinct codewords. For the linear code $C = \ker(H)$, it equals the minimum nonzero weight:

$$\boxed{d_{\min} = \min_{\substack{v \in C \\ v \neq 0}} \mathrm{wt}(v).}$$

It is useful to give the quantity on the right a temporary name:

$$w_{\min} := \min_{\substack{v \in C \\ v \neq 0}} \mathrm{wt}(v).$$

We prove equality in two directions.

First, let $c, c' \in C$ be any distinct codewords. Since H is linear,

$$H(c' - c) = Hc' - Hc = 0.$$

Thus $c' - c$ is a codeword, and it is nonzero because $c' \neq c$. By the definition of $w_{\min}$,

$$d_H(c, c') = \mathrm{wt}(c' - c) \geq w_{\min}.$$

Every distance appearing in the definition of $d_{\min}$ is therefore at least $w_{\min}$, so

$$d_{\min} \geq w_{\min}.$$

For the other direction, choose a nonzero codeword v with $\mathrm{wt}(v) = w_{\min}$. Such a vector exists because C is finite. The zero vector also belongs to C, since $H0 = 0$. Hence 0 and v are an allowed pair in the definition of minimum distance, and

$$d_H(0, v) = \mathrm{wt}(v) = w_{\min}.$$

The minimum over all pairs cannot exceed the distance of this particular pair. Therefore,

$$d_{\min} \leq w_{\min}.$$

The two inequalities give the claimed equality.

The fact that differences of codewords remain in C supplies the first direction. The fact that every nonzero codeword can itself be written as $v - 0$ supplies the second. Both are needed to justify equality.

Combining this equality with (8.4) gives

$$\boxed{d_{\min} = \min_{\substack{v \in C \\ v \neq 0}} \operatorname{wt}(v) = \min_{\substack{e \neq 0 \\ He = 0}} \operatorname{wt}(e).} \tag{9.2}$$

The three expressions answer the same question in different forms: the closest separation of two valid words, the lightest nonzero vector in the code, and the smallest nonzero error that the checks can miss.

This immediately determines the guaranteed detection limit. If

$$1 \leq \operatorname{wt}(e) < d_{\min},$$

then He cannot be zero, because that would contradict the last minimum. Thus every error affecting at most $d_{\min} - 1$ positions is detected. At weight $d_{\min}$, there is at least one undetectable error: any minimum-weight nonzero codeword used as an error pattern. Some other errors of that weight or greater may still be detected; the statement concerns what is guaranteed for every error up to a given size.

10 Reading Minimum Distance from the Columns of the Check Matrix

Writing H in columns as in (2.1),

$$He = e_1 h_1 + \cdots + e_n h_n. \tag{10.1}$$

For a binary error pattern, the coefficients e_j select precisely the flipped positions. For example, if only positions i and j flip, then $He = h_i + h_j$. This is a linear combination of columns whether or not it equals zero. The occurrence of an error alone does not establish linear dependence.

If the error is both nonzero and undetectable, however, then

$$e_1 h_1 + \cdots + e_n h_n = 0$$

with coefficients that are not all zero. This is exactly a nontrivial linear relation among the columns corresponding to its nonzero entries.

Let ℓ be the smallest number of columns of H that form a linearly dependent set. Then $\ell = d_{\min}$.

First, choose an undetectable error of weight $d_{\min}$, whose existence follows from (9.2). Its nonzero entries give a linear dependence among $d_{\min}$ columns. Therefore,

$$\ell \leq d_{\min}.$$

Conversely, suppose the columns $h_{j_1}, \ldots, h_{j_\ell}$ are linearly dependent. There are coefficients $a_1, \ldots, a_\ell \in \mathbb{F}_2$, not all zero, such that

$$a_1 h_{j_1} + \cdots + a_\ell h_{j_\ell} = 0.$$

Define an error vector by putting $e_{j_r} = a_r$ for these positions and putting zero in every other position. Then

$$e \neq 0, \qquad He = 0, \qquad \operatorname{wt}(e) \leq \ell.$$

The last inequality is written as "at most" because coefficients in a linear dependence are allowed to be zero. Since $d_{\min}$ is the smallest nonzero undetectable error weight,

$$d_{\min} \leq \operatorname{wt}(e) \leq \ell.$$

Together, the bounds prove

$$\boxed{\begin{array}{c} d_{\min} = \text{the smallest number of columns of } H \\ \text{that form a linearly dependent set.} \end{array}} \tag{10.2}$$

For our Hamming matrix, this number is 3:

- One column cannot be dependent, because no column is zero.
- Two distinct columns cannot be dependent over $\mathbb{F}_2$. The only possible nontrivial relations would require one column to be zero or the two columns to be equal, and both possibilities were excluded.
- Three dependent columns exist. The labels of positions 1, 2, and 3 satisfy $h_1 + h_2 = h_3$, so $h_1 + h_2 + h_3 = 0$.

Hence

$$d_{\min} = 3. \tag{10.3}$$

Every one-bit or two-bit error gives a nonzero syndrome, while the three-bit error $u_1 + u_2 + u_3$ is undetectable.

Detecting every two-bit error does not mean that the single-error correction rule can recognize an error as a two-bit error. Its syndrome is nonzero, but it can coincide with the syndrome of a single-bit error. Correction requires different allowed errors to have different syndromes.

11 Why Correcting Up to t Errors Requires Minimum Distance Greater Than 2t

Let t be a nonnegative integer representing the maximum number of flipped positions we want to correct. The allowed error patterns form the set

$$\mathcal{E}_t := \{e \in \mathbb{F}_2^n : \mathrm{wt}(e) \leq t\}. \tag{11.1}$$

(The letter $\mathcal{E}$ refers to *errors*.) This set includes 0, because the transmission may contain no error.

Given y and $s = Hy$, we seek an error $e \in \mathcal{E}_t$ satisfying

$$He = s.$$

Once it is determined, the original codeword is recovered as

$$\widehat{c} = y - e. \tag{11.2}$$

The hat marks the decoder's estimate. Under the assumed error bound, the actual error is a solution, so existence is guaranteed. Correcting uniquely requires that there be only one allowed solution.

Thus we ask whether H assigns different syndromes to different vectors in $\mathcal{E}_t$. This is injectivity on a restricted set of errors, not injectivity on the entire space $\mathbb{F}_2^n$. The set $\mathcal{E}_t$ need not be a vector subspace.

Why would two allowed solutions cause a real ambiguity? If $e \neq e'$ both satisfy $He = He' = Hy$, then

$$H(y - e) = 0, \qquad H(y - e') = 0.$$

So $y - e$ and $y - e'$ are distinct valid codewords, either of which could have produced the same received word with at most t bit flips. Knowing the whole received word does not resolve this ambiguity.

Unique correction of every error in $\mathcal{E}_t$ is possible exactly when

$$d_{\min} > 2t.$$

Sufficiency: separation greater than $2t$ prevents ambiguity. Suppose $d_{\min} > 2t$, and suppose two allowed errors have the same syndrome:

$$He = He', \qquad \mathrm{wt}(e) \le t, \qquad \mathrm{wt}(e') \le t.$$

Linearity gives

$$H(e - e') = 0.$$

If $e \neq e'$, then $e - e'$ is a nonzero codeword, so

$$\mathrm{wt}(e - e') \ge d_{\min}.$$

At the same time, a coordinate of $e - e'$ can be nonzero only if at least one of the corresponding coordinates of e and e' is nonzero. Its nonzero positions therefore lie in the union of the two sets of error positions. The union has at most the sum of their sizes:

$$\mathrm{wt}(e - e') \le \mathrm{wt}(e) + \mathrm{wt}(e') \le 2t.$$

Overlapping errors may cancel in the difference, but cannot increase this bound.

We would therefore have

$$d_{\min} \le \mathrm{wt}(e - e') \le 2t,$$

contradicting $d_{\min} > 2t$. Thus $e = e'$, and the syndrome identifies exactly one allowed error.

The factor 2 appears because we are comparing two candidate explanations of a received word. Each candidate can involve up to t positions, so their difference can involve up to $2t$ positions.

Necessity: separation at most $2t$ creates an ambiguous received word. Suppose instead that $d_{\min} \le 2t$. Choose a nonzero codeword v of weight $d_{\min}$. Partition its nonzero positions into two disjoint sets, each containing at most t positions. This is possible because there are at most $2t$ positions altogether.

Let e agree with v on the first set and be zero elsewhere, and let e' agree with v on the second set and be zero elsewhere. Then

$$v = e + e', \qquad \mathrm{wt}(e) \le t, \qquad \mathrm{wt}(e') \le t.$$

Because $v \neq 0$, the two error vectors are distinct. Moreover,

$$0 = Hv = He + He',$$

so they have the same syndrome.

Now take the received word to be $y = e$. It can arise in two ways:

$$\begin{aligned} y &= 0 + e \quad \text{from transmitting the codeword } 0, \\ y &= v + e' \quad \text{from transmitting the codeword } v. \end{aligned}$$

The second equality holds because

$$v + e' = (e + e') + e' = e.$$

Both transmissions satisfy the bound of at most t errors, yet they produce the same received word from different original codewords. No decoder using only that word can always recover the correct original in both cases.

This proves both directions:

$$\boxed{\begin{array}{c}\text{Every error affecting at most } t \text{ positions can be uniquely corrected} \\ \iff \quad d_{\min} > 2t.\end{array}} \tag{11.3}$$

Since the quantities are integers, this is equivalent to $d_{\min} \geq 2t + 1$. The largest guaranteed correction radius is therefore

$$t_{\max} = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor, \tag{11.4}$$

where $\lfloor a \rfloor$ denotes the largest integer at most a.

For our Hamming code, (10.3) and (11.4) give $t_{\max} = 1$. The rule (4.4) achieves this guarantee; no decoder can correct every error affecting at most two bits.

12 Hamming Balls as Sets of Possible Received Words

Fix a center $c \in \mathbb{F}_2^n$. For any word $y \in \mathbb{F}_2^n$, let $v = y - c$ be its **displacement vector** from c. Then

$$y = c + v, \qquad d_H(c, y) = \mathrm{wt}(v). \tag{12.1}$$

For an integer $0 \leq t \leq n$, define

$$\begin{aligned} B_t(c) &:= \{y \in \mathbb{F}_2^n : d_H(c, y) \leq t\} \\ &= \{c + v : v \in \mathbb{F}_2^n,\ \mathrm{wt}(v) \leq t\}. \end{aligned} \tag{12.2}$$

This is the **closed Hamming ball** of radius t centered at c (B stands for *ball*). The words in a ball need not be codewords. When $c \in C$ is transmitted and y is received, the displacement $v = y - c$ is the error vector e in (3.1), and the ball consists of the possible received words under the bound of at most t bit flips.

A **Hamming sphere** uses equality instead:

$$S_t(c) := \{y \in \mathbb{F}_2^n : d_H(c, y) = t\}. \tag{12.3}$$

Here S stands for *sphere*. The sphere contains words that differ from the center in exactly t positions; the ball contains words that differ in any number of positions from zero through t. We use "sphere" and "ball" with this distinction throughout this tutorial.

For the counting argument, let $r \in \{0, 1, \ldots, n\}$ denote the ball radius. To count these words, first take $n = 3$ and $c = 000$. List the positions that differ from c and the resulting word:

Number i of differing positions	Sets of positions	Resulting words y	Count
0	$\varnothing$	000	1
1	$\{1\}, \{2\}, \{3\}$	$100, 010, 001$	3
2	$\{1,2\}, \{1,3\}, \{2,3\}$	$110, 101, 011$	3
3	$\{1,2,3\}$	111	1

For example, selecting positions 1 and 3 gives

$$v = u_1 + u_3 = 100 + 001 = 101, \qquad y = c + v = 000 + 101 = 101,$$

so $d_H(000, y) = \mathrm{wt}(101) = 2$. Once the positions are fixed, the resulting word is fixed: a changed binary digit has only one possible new value.

For arbitrary n and $1 \le i \le n$, there are $n(n-1)\cdots(n-i+1)$ ordered selections of i distinct positions. Each set of positions is counted $i!$ times, once for each order, so

$$\#\{\text{sets of } i \text{ positions}\} = \frac{n(n-1)\cdots(n-i+1)}{i!} = \frac{n!}{i!(n-i)!} = \binom{n}{i}. \tag{12.4}$$

For a fixed center c, these sets correspond exactly to words at distance i:

$$I \subseteq \{1, \ldots, n\}, \quad |I| = i \quad \longleftrightarrow \quad y = c + \sum_{j \in I} u_j.$$

The correspondence is one-to-one because I can be recovered as $\{j : y_j \neq c_j\}$. Therefore,

$$|S_i(c)| = \binom{n}{i}. \tag{12.5}$$

This also holds for $i = 0$: the empty set gives the unchanged word c, so $|S_0(c)| = \binom{n}{0} = 1$.

A ball includes every distance from zero through its radius. In the three-bit example,

$$B_2(000) = \underbrace{\{000\}}_{\text{distance } 0} \cup \underbrace{\{100, 010, 001\}}_{\text{distance } 1} \cup \underbrace{\{110, 101, 011\}}_{\text{distance } 2},$$
$$|B_2(000)| = 1 + 3 + 3 = 7.$$

In general, $B_r(c) = S_0(c) \cup \cdots \cup S_r(c)$.

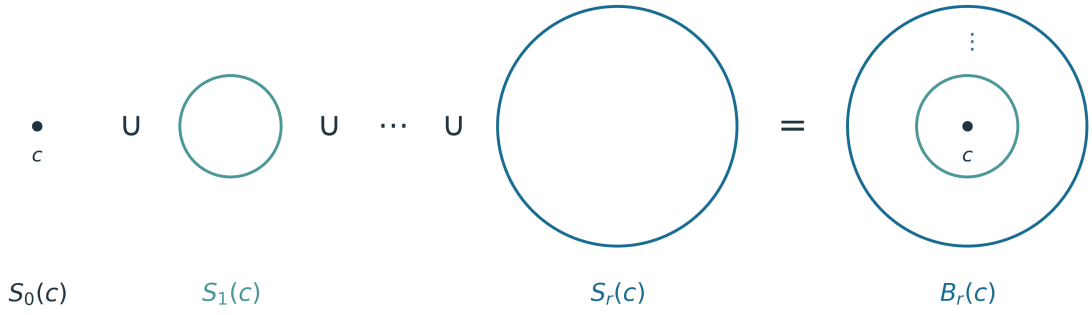


Figure 1: A ball as a union of distance layers. The center is $S_0(c) = \{c\}$. Combining the spheres $S_1(c), \ldots, S_r(c)$ with this center gives $B_r(c)$. The circular guides represent the discrete layers of integer Hamming distance; the dots indicate omitted layers. The drawing records the union of these sets, not a continuous Euclidean volume.

A word has only one distance from c, so these sets are disjoint and their counts add:

$$\boxed{V(n, r) := |B_r(c)| = \sum_{i=0}^{r} \binom{n}{i}.} \tag{12.6}$$

The **volume** $V(n, r)$ counts the words in the ball and is independent of its center. The ball $B_r(c)$ is a set of words and need not be a vector subspace. In particular,

$$\begin{aligned} B_0(c) &= \{c\}, & V(n,0) &= 1, \\ B_1(c) &= \{c, c+u_1, \ldots, c+u_n\}, & V(n,1) &= 1+n, \\ & & V(n,2) &= 1+n+\binom{n}{2}. \end{aligned}$$

The term $1 = \binom{n}{0}$ counts the unchanged word, allowing for no error. At the largest possible radius,

$$B_n(c) = \mathbb{F}_2^n, \qquad V(n,n) = \sum_{i=0}^{n} \binom{n}{i} = (1+1)^n = 2^n. \tag{12.7}$$

These balls consist of discrete bit strings; their geometric interpretation comes from counting differing positions.

Suppose the balls around two distinct codewords c, c' overlap. A word y in their intersection satisfies

$$d_H(c, y) \le t, \qquad d_H(c', y) \le t.$$

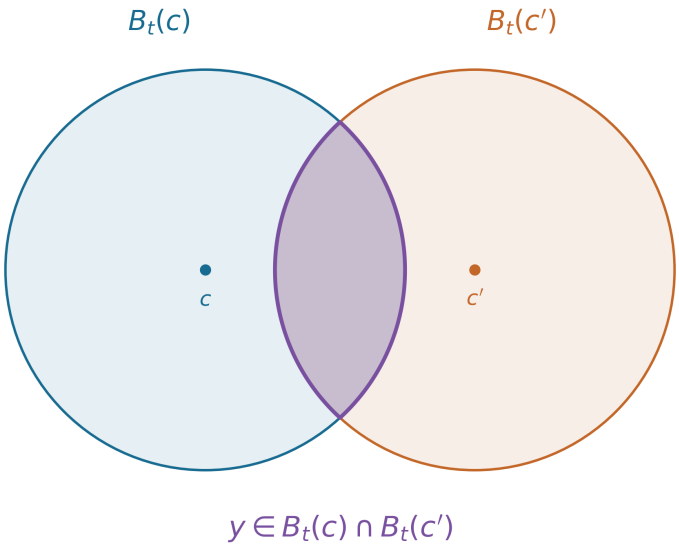


Figure 2: An arbitrary word in the intersection. The purple region and its solid outline represent $B_t(c) \cap B_t(c')$. Any word y in this set satisfies both distance inequalities, with equality allowed. No particular location is assigned to y. The round shapes illustrate membership in the closed balls; the actual sets consist of discrete binary words.

It could have arisen from either codeword within the permitted error bound. Therefore unique correction of every allowed error is equivalent to the balls around distinct codewords being disjoint.

The triangle inequality (7.5) makes the separation requirement visible. If such a shared word existed, then

$$d_H(c, c') \le d_H(c, y) + d_H(y, c') \le 2t. \tag{12.8}$$

Thus $d_{\min} > 2t$ prevents any intersection. Conversely, the necessity proof of (11.3) supplies a word in two distinct codeword balls whenever $d_{\min} \le 2t$.

The algebraic and geometric arguments describe the same ambiguity. Algebraically, two allowed errors satisfy $He = He'$. Geometrically, two codeword balls contain the same received word. In either description, correction fails because one observation has two permitted explanations.

Counting the disjoint balls using (12.6) gives the **Hamming bound**, also called the sphere-packing bound. For $0 \le t \le n$, any binary code of length n that corrects every error of weight at most t satisfies

$$\boxed{|C|\,V(n,t) = |C| \sum_{i=0}^{t} \binom{n}{i} \le 2^n.} \tag{12.9}$$

For a linear code with k information bits and $m = n - k$ independent checks, $|C| = 2^{n-m}$, so

$$\sum_{i=0}^{t} \binom{n}{i} \le 2^m, \qquad m \ge \left\lceil \log_2 \left(\sum_{i=0}^{t} \binom{n}{i} \right) \right\rceil. \tag{12.10}$$

Here $\lceil a \rceil$ is the smallest integer at least a. Equivalently, the 2^m possible syndromes must distinguish all $V(n,t)$ allowed error patterns. This is a necessary counting condition; the correction guarantee still requires $d_{\min} \ge 2t + 1$. If the disjoint balls cover all 2^n words, equality holds and the code is called **perfect** for radius t.

13 A Three-Bit Example Bringing the Arguments Together

The case $m = 2$ gives a small example in which we can list every codeword and every one-bit error. Keeping the binary position-label order from Section 1,

$$h_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \qquad h_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad h_3 = \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

and

$$H = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}. \tag{13.1}$$

The conditions $Hc = 0$ say

$$c_2 + c_3 = 0, \qquad c_1 + c_3 = 0.$$

Thus all three coordinates agree, and

$$C = \{000, 111\}. \tag{13.2}$$

This is the three-bit repetition code, also the $m = 2$ instance of our Hamming construction.

There is only one nonzero codeword, 111, and its weight is 3. Equivalently, the distance between the two codewords is 3, and all three columns of H are needed to obtain a dependence. Therefore,

$$d_{\min} = 3.$$

If 111 is sent and received unchanged, there is no error, despite the codeword having weight 3. If 000 is sent and the error is 111, the received word is 111, and the error is undetectable. This illustrates (8.4): the nonzero codeword 111 can also serve as an undetectable error.

The syndrome table for all errors of weight at most one is:

Error pattern e	Number of flipped positions $\mathrm{wt}(e)$	Syndrome He
000	0	00
100	1	01
010	1	10
001	1	11

These syndromes are all different, so the error can be identified under the one-error bound. The corresponding balls are

$$B_1(000) = \{000, 100, 010, 001\}, \tag{13.3}$$

$$B_1(111) = \{111, 011, 101, 110\}. \tag{13.4}$$

They are disjoint. For example, receiving 100 under this bound identifies 000 as the transmitted word: 100 lies one flip from 000 but two flips from 111.

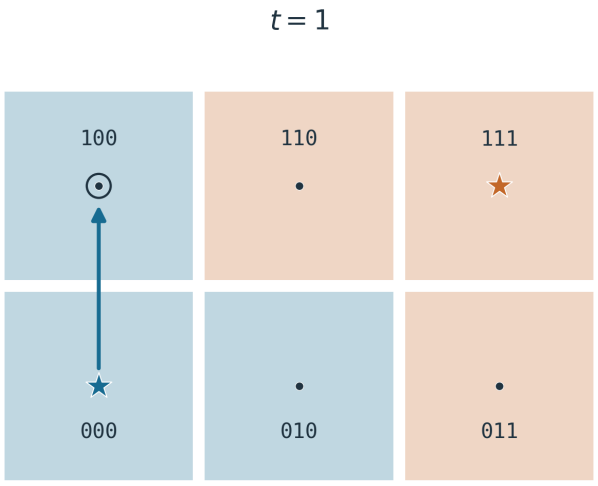


Figure 3: The intersections of $B_1(000)$ and $B_1(111)$ with a six-word grid. Stars mark codewords, and a colored tile represents a word within one flip of the star of that color. The ringed word 100 belongs only to the blue ball. Each horizontal or vertical step flips one bit, and distances between displayed words are exact. The omitted words are 001 and 101; each full ball contains $V(3, 1) = 4$ words. Colors identify codeword centers.

The sphere around 000 is

$$S_1(000) = \{100, 010, 001\};$$

the ball also contains its center 000, allowing for the possibility of no error.

Now allow up to two errors. The same received word has two permitted explanations:

$$100 = 000 + 100 = 111 + 011. \tag{13.5}$$

One explanation uses a one-bit error, the other a two-bit error. Both produce syndrome 01, since

$$H(100) = h_1, \qquad H(011) = h_2 + h_3 = h_1.$$

The word 100 lies in both $B_2(000)$ and $B_2(111)$, so unique correction is no longer guaranteed.

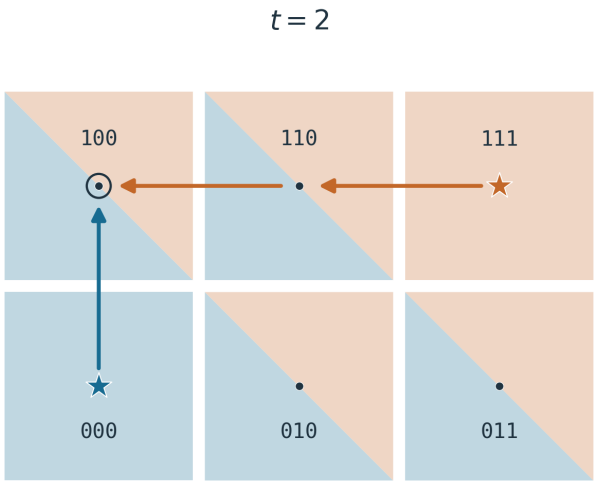


Figure 4: The same grid with radius two. A split tile belongs to both $B_2(000)$ and $B_2(111)$. The arrows show $000 \to 100$ and $111 \to 110 \to 100$: two permitted origins of the same received word. Each star remains outside the other star's ball, since the codewords are three flips apart. Every nonzero error of weight at most two is detectable, but correction up to two errors is not guaranteed. Each full ball contains $V(3, 2) = 7$ words.

The syndrome is still nonzero: the corruption is detectable. If the decoder nevertheless assumes a single error, it interprets syndrome 01 as a flip in position 1 and outputs 000. That output is correct for the first transmission and incorrect for the second. A nonzero syndrome establishes a violation of the checks; recovering the original word additionally requires an error bound under which the possible explanations are unique.

14 Word Length, Redundancy, and Correction Regions

Recall the parameters: n is the total word length, m is the number of independent parity checks and syndrome bits, and $k = \dim C$ is the number of information bits. For the Hamming family,

$$n = 2^m - 1, \qquad k = n - m, \qquad d_{\min} = 3. \tag{14.1}$$

Equations (9.2) and (11.4) give the guaranteed limits

$$\text{detection: } d_{\min} - 1, \qquad \text{correction: } \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor. \tag{14.2}$$

Check bits m	Word length n	Information bits k	Minimum distance	Guaranteed detection	Guaranteed correction
2	3	1	3	up to 2 bit flips	up to 1 bit flip
3	7	4	3	up to 2 bit flips	up to 1 bit flip
4	15	11	3	up to 2 bit flips	up to 1 bit flip
5	31	26	3	up to 2 bit flips	up to 1 bit flip

Increasing m supplies more distinct position labels and permits longer words. The minimum distance stays 3, so the guaranteed correction radius stays 1. Detection and correction are separate guarantees: the single-error decoder can miscorrect a two-bit error.

Assign one color to each codeword c and use that color for $B_1(c)$. Equations (12.2) and (12.6) give

$$B_1(c) = \{c, c + u_1, \ldots, c + u_n\}, \qquad |B_1(c)| = 1 + n.$$

These balls are disjoint because $d_{\min} = 3$. Since $n + 1 = 2^m$,

$$\boxed{|C|\,|B_1(c)| = 2^{n-m}(1+n) = 2^{n-m}2^m = 2^n.} \tag{14.3}$$

For the seven-bit code, this is $16 \times 8 = 128$: 16 codewords, 8 words per ball, and 128 words in the full space. The disjoint balls exhaust the space, so Hamming codes are perfect for radius one. Every received word belongs to exactly one such ball, although two or more errors can place it in the ball of a different codeword.

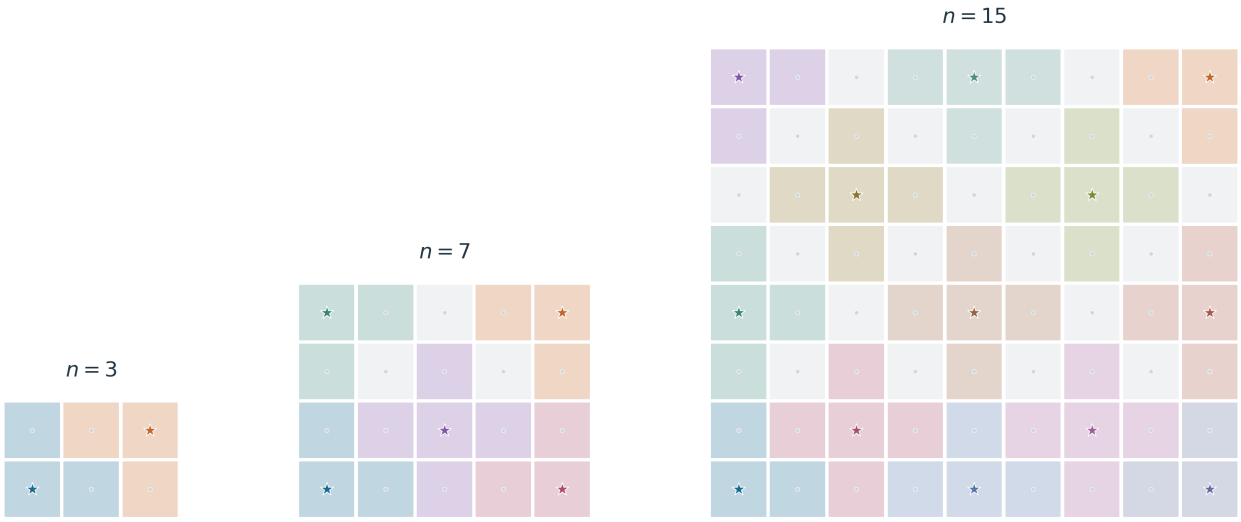


Figure 5: Each tile represents one word, and each star marks a codeword. A tile has the star's color if its word belongs to $B_1(c)$ for that codeword c. A grey tile belongs to a ball whose center is not shown. All three panels use the same tile size and show only part of the full space:

Word length n	Words shown in the grid	Total words in $\mathbb{F}_2^n$
3	6	8
7	20	128
15	72	32,768

Let $v_{i,j}$ be the word in column i and row j, counting from zero at the bottom-left corner. Thus i counts steps to the right, j counts steps upward, and $v_{0,0} = 0$.

Set $a = \lfloor n/2 \rfloor$. The grid has $0 \le i \le n - a$ and $0 \le j \le a$. Each step changes one bit:

- From row j to row $j + 1$, flip bit $j + 1$.
- From column i to column $i + 1$, flip bit $a + i + 1$.

Vertical steps use the first a bit positions; horizontal steps use the remaining positions. Two grid words therefore differ in $|j - j'|$ positions from the first group and $|i - i'|$ from the second:

$$d_H(v_{i,j}, v_{i',j'}) = |i - i'| + |j - j'|. \tag{14.4}$$

Thus grid distance equals Hamming distance for the displayed words. As n grows, the full space has more words and a larger maximum distance n, while the Hamming code's minimum distance remains 3.

Now fix $n = 7$ and compare the Hamming code with the repetition code

$$C_{\text{rep}} = \{0000000, 1111111\}.$$

The repetition code has dimension 1 and minimum distance 7, so it detects every error affecting at most six bits and uniquely corrects every error affecting at most three bits. Its larger minimum distance accompanies fewer codewords and fewer information bits.

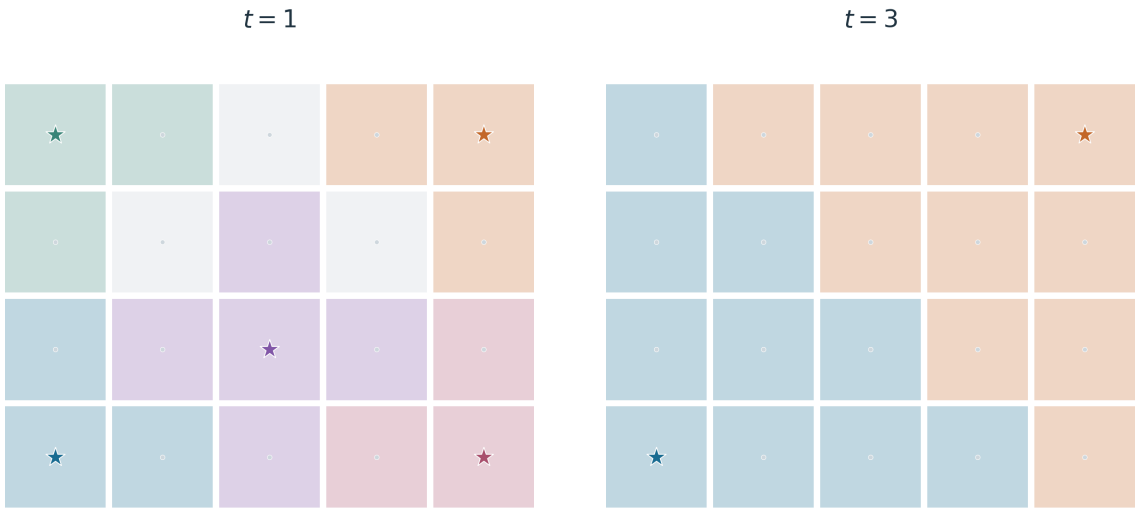


Figure 6: The same 20-word grid for two seven-bit codes. Left: Hamming code with radius-one balls. Right: repetition code with radius-three balls. Stars mark the codewords within the grid. Grey words on the left have their radius-one centers outside the grid. The larger correction radius on the right follows from greater minimum distance between codewords.

Seven-bit code	Information bits k	Redundant bits m	Minimum distance	Guaranteed detection	Guaranteed correction
Hamming	4	3	3	up to 2 bit flips	up to 1 bit flip
Repetition	1	6	7	up to 6 bit flips	up to 3 bit flips

Their full balls satisfy the same counting bound at different radii:

$$16\,V(7,1) = 16(1+7) = 128, \qquad 2\,V(7,3) = 2(1+7+21+35) = 128. \tag{14.5}$$

Detection depends on the distance from one codeword to another; unique correction depends on the disjointness of the allowed-error balls. Assigning a shared word to one color cannot remove the alternative permitted origin.

There is also a graph interpretation. Take the words of $\mathbb{F}_2^n$ as vertices and join two vertices when they differ in one bit. This is the hypercube graph Q_n, whose shortest-path distance is d_H; code design becomes the selection of separated vertices, and correction becomes packing disjoint graph balls.